GIT & GITHUB

GIT:

- GIT IS A OPEN SOURCE VERSION CONTROL SYSTEM USED TO MANAGE AN TRACK CHANGES IN SOFTWARE DVELOPMENT PROJECTS.
- IT ALLOWS MULTIPLE DEVELOPERS TO COLLABORATE ON A PROJECT BY KEEPING TRACK OF CODE CHANGES,MANAGING,DIFFERENT VERSIONS,AND FACILITATING TEAMWORK.

VERSION CONTROL:

- GIT TRACKS CHANGES TO FILES,ALLOWING DEVELOPERS TO ACCESS PREVIOUS VERSIONS,IDENTIFY BUGS,AND REVERT TO EARLIER STATES WHEN NEEDED.

COLLABORATION:

- GIT ALLOWS MULTIPLE DEVELOPERS TO WORK ON THE SAME PROJECT SIMULTANEOUSLY WITHOUT INTERFERING WITH EACH OTHERS CODE.
- CHANGES CAN BE MERGED SEAMLESSLY.

BRANCHING AND MERGING:

- DEVELOPERS CAN CREATE BRANCHES TO WORK ON SPECIFIC FEATURES OR FIXES INDEPENDENTLY
- THESE BRACHES CAN BE MERGED BACK INTO THE MAIN CODEBASE WHEN READY.

CODE HISTORY AND TRACKING:

- GIT PROVIDES A DETAILED HISTORY OF WHO MADE WHAT CHANGES AND WHEN,HELPING IN DEBUGGING AND ACCOUNTABILITY.

BACKUP AND RECOVERY:

- BY PUSHING CODE TO AREMOTE REPOSITORY LIKE GITHUB ,DEVELOPERS ENSURE THEIR WORK IS BACKED UP AND RECOVERABLE.

EFFICIENT WORKFLOW:

- TOOLS AND COMMANDS IN GIT STREAMLINE DEVELOPMENT TASKS SUCH AS CODE REVIEWS,FEATURE DEVELOPMENT,AND RELEASES.

OPEN SOURCE AND WIDELY SUPPORTED:

- GIT IS FREE,OPEN SOURCE,AND COMPATIBLE WITH ALL MAJOR DEVELOPMENT TOOLS AND OPERATING SYSTEMS.

GITHUB:

- **GITHUB** IS A **WEB-BASED PLATFORM** FOR **HOSTING AND MANAGING GIT REPOSITORIES**. IT PROVIDES TOOLS FOR **VERSION CONTROL, COLLABORATION, AND PROJECT MANAGEMENT**, MAKING IT EASIER FOR DEVELOPERS TO WORK TOGETHER ON SOFTWARE PROJECTS.

HOSTING GIT REPOSITORIES

- STORES CODE ONLINE WITH EASY ACCESS.

COLLABORATION

- ENABLES TEAMS TO WORK TOGETHER USING FEATURES LIKE PULL REQUESTS AND ISSUE TRACKING.

PROJECT MANAGEMENT

- PROVIDES TOOLS LIKE KANBAN BOARDS, TASK MANAGEMENT, AND ISSUE TRACKING.

OPEN-SOURCE CONTRIBUTIONS

- ALLOWS DEVELOPERS TO CONTRIBUTE TO PUBLIC PROJECTS AND FORK REPOSITORIES.

CI/CD (CONTINUOUS INTEGRATION & DEPLOYMENT)

- AUTOMATES BUILDING, TESTING, AND DEPLOYMENT WITH GITHUB ACTIONS.

CODE REVIEW

- FACILITATES TEAM-BASED CODE REVIEWS AND DISCUSSIONS.

SECURITY & CODE SCANNING

- OFFERS SECURITY FEATURES LIKE DEPENDENCY VULNERABILITY SCANNING.

CLOUD DEVELOPMENT

- PROVIDES GITHUB CODESPACES FOR CODING DIRECTLY IN THE CLOUD.

PORTFOLIO SHOWCASE

- ALLOWS DEVELOPERS TO DISPLAY THEIR WORK PUBLICLY.

INTEGRATION WITH DEV TOOLS

- WORKS WITH VARIOUS DEVOPS, TESTING, AND MONITORING TOOLS.

HISTORY OF GIT & GITHUB

HISTORY OF GIT

- GIT WAS CREATED BY **LINUS TORVALDS** IN **2005** TO MANAGE THE DEVELOPMENT OF THE LINUX KERNEL.
- BEFORE GIT, THE LINUX COMMUNITY USED A PROPRIETARY VERSION CONTROL SYSTEM CALLED **BITKEEPER**.
- DUE TO DISPUTES BETWEEN THE LINUX COMMUNITY AND BITKEEPER'S COMPANY, THE NEED AROSE FOR A NEW, OPEN-SOURCE VERSION CONTROL SYSTEM.
- LINUS TORVALDS DESIGNED GIT TO BE **FAST, DISTRIBUTED, AND SCALABLE** FOR LARGE PROJECTS.
- GIT BECAME POPULAR QUICKLY AND WAS ADOPTED BY MANY OPEN-SOURCE AND ENTERPRISE PROJECTS.
- OVER THE YEARS, GIT HAS EVOLVED WITH IMPROVEMENTS IN PERFORMANCE, SECURITY, AND USER EXPERIENCE.
- TODAY, GIT IS THE MOST WIDELY USED VERSION CONTROL SYSTEM IN SOFTWARE DEVELOPMENT.

HISTORY OF GITHUB

- GITHUB WAS FOUNDED IN **2008** BY **TOM PRESTON-WERNER, CHRIS WANSTRATH, PJ HYETT, AND SCOTT CHACON** AS A HOSTING PLATFORM FOR GIT REPOSITORIES.
- IT PROVIDED A **USER-FRIENDLY INTERFACE**, **COLLABORATION TOOLS**, AND **CLOUD STORAGE** FOR GIT PROJECTS.
- GITHUB POPULARIZED FEATURES LIKE **PULL REQUESTS, ISSUES, AND FORKING**, MAKING OPEN-SOURCE DEVELOPMENT MORE ACCESSIBLE.
- IN 2011, GITHUB REACHED **1 MILLION REPOSITORIES**, SHOWING ITS RAPID GROWTH.
- IN 2018, MICROSOFT **ACQUIRED GITHUB FOR $7.5 BILLION**, INTEGRATING IT INTO ITS CLOUD AND DEVELOPER ECOSYSTEM.
- GITHUB INTRODUCED NEW FEATURES LIKE **GITHUB ACTIONS (CI/CD), GITHUB COPILOT (AI PAIR PROGRAMMING), AND GITHUB CODESPACES (CLOUD DEVELOPMENT ENVIRONMENT)**.
- TODAY, GITHUB IS THE **LARGEST CODE HOSTING PLATFORM**, WITH MILLIONS OF DEVELOPERS AND COMPANIES USING IT WORLDWIDE.

DIFFERENCE BETWEEN GIT & GITHUB:

GIT

- GIT IS A DISTRIBUTED VERSION CONTROL SYSTEM.
- GIT WORKS LOCALLY ON A USER'S MACHINE.
- GIT ALLOWS USERS TO TRACK CHANGES, BRANCH, AND MERGE CODE.
- GIT CAN BE USED WITHOUT GITHUB.
- GIT IS A COMMAND-LINE TOOL.
- GIT DOES NOT REQUIRE AN INTERNET CONNECTION FOR LOCAL OPERATIONS.
- GIT IS MAINTAINED BY THE OPEN SOURCE COMMUNITY.
- GIT IS FREE AND OPEN SOURCE.
- GIT ALLOWS USERS TO CREATE REPOSITORIES LOCALLY.
- GIT USES COMMANDS LIKE `GIT INIT`, `GIT CLONE`, AND `GIT COMMIT`.

GITHUB

- GITHUB IS A WEB-BASED PLATFORM FOR HOSTING GIT REPOSITORIES.
- GITHUB STORES REPOSITORIES IN THE CLOUD.
- GITHUB PROVIDES ADDITIONAL FEATURES LIKE ISSUE TRACKING, COLLABORATION TOOLS, AND PULL REQUESTS.
- GITHUB REQUIRES GIT FOR VERSION CONTROL.
- GITHUB PROVIDES A GRAPHICAL USER INTERFACE ALONG WITH COMMAND-LINE SUPPORT.
- GITHUB REQUIRES AN INTERNET CONNECTION TO ACCESS AND MANAGE REPOSITORIES.
- GITHUB IS OWNED BY MICROSOFT.
- GITHUB OFFERS BOTH FREE AND PAID PLANS WITH ADDITIONAL FEATURES.
- GITHUB PROVIDES CLOUD STORAGE FOR REPOSITORIES.
- GITHUB USES FEATURES LIKE PULL REQUESTS, FORKS, AND ACTIONS.

GIT COMMANDS:

- GIT CONFIG –GLOBAL USER.NAME”YOUR NAME”
  SET YOUR USERBAME GLOBALLY.

- GIT CONFIG -GLOBAL USER.EMAIL”YOUR EMAIL”
  SET YOUR EMAIL GLOBALLY.

- GIT CLONE<REPOSITORY URL>
  THIS CLONES A REPOSITORY FROM A REMOTE SERVER TO YOUR LOCAL MACHINE.

- GIT INIT
  INITIALIZES A NEW GIT REPOSITORY IN THE CURRENT DIRECTORY.

- GIT PULL
  FETCHES CHANGES FROM THE REMOTE REPOSITORY AND MERGES THEM INTO THE CURRENT BRANCH.

- GIT ADD
  ADDS ALL MODIFIED AND NEW FILES TO THE STAGING AREA.

- GIT COMMIT –M
  CREATES A NEW COMMIT WITH THE CHANGES IN THE STAGING AREA AND SPECIFIES THE COMMIT MESSAGE INLINE.

- GIT PUSH
  PUSHES LOCAL COMMITS TO THE REMOTE REPOSITORY.

- GIT ADD.
  STAGES ALL CHANGED FILES FOR COMMITTING.

- GIT STATUS
  SHOWS THE STATUS OF CHANGES (TRACKED, UNTRACKED, MODIFIED FILES).

- GIT LOG
  DISPLAYS THE HISTORY OF COMMITS IN THE REPOSITORY.

- GIT BRANCH
  LISTS ALL BRANCHES IN THE REPOSITORY.

- GIT BRANCH[BRANCH_NAME]
  MERGES A SPECIFIED BRANCH INTO THE CURRENT BRANCH.

- GIT CHECKOUT[BRANCH_NAME]
  SWITCHES TO A DIFFERENT BRANCH.

- GIT PUSH
  PUSHES LOCAL CHANGES TO A REMOTE REPOSITORY.

- GIT REMOTE –V
  DISPLAYS THE REMOTE REPOSITORIES ASSOCIATED WITH YOUR PROJECT.

- GIT STASH
  TEMPORARILY SAVES UNCOMMITTED CHANGES WITHOUT COMMITTING THEM.

```
Pallavi Bandreddy@DESKTOP-POOHEON MINGW64 ~
$ pwd
/c/Users/hp

Pallavi Bandreddy@DESKTOP-POOHEON MINGW64 ~
$ mkdir gitcommands

Pallavi Bandreddy@DESKTOP-POOHEON MINGW64 ~
$ cd gitcommands

Pallavi Bandreddy@DESKTOP-POOHEON MINGW64 ~/gitcommands
$ git config --global user.name"Pallavibandredddy"

Pallavi Bandreddy@DESKTOP-POOHEON MINGW64 ~/gitcommands
$ git config --global user.email"pallavibandreddy6@gmail.com"

Pallavi Bandreddy@DESKTOP-POOHEON MINGW64 ~/gitcommands
$ git init
Initialized empty Git repository in C:/Users/hp/gitcommands/.git/

Pallavi Bandreddy@DESKTOP-POOHEON MINGW64 ~/gitcommands (master)
$ echo "hii evryone" >commands.txt

Pallavi Bandreddy@DESKTOP-POOHEON MINGW64 ~/gitcommands (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        commands.txt

nothing added to commit but untracked files present (use "git add" to track)

Pallavi Bandreddy@DESKTOP-POOHEON MINGW64 ~/gitcommands (master)
$ git add commands.txt
warning: in the working copy of 'commands.txt', LF will be replaced by CRLF the next time Git touches
t

Pallavi Bandreddy@DESKTOP-POOHEON MINGW64 ~/gitcommands (master)
$ git commit -m "initial commit"
[master (root-commit) 1aadc9f] initial commit
 1 file changed, 1 insertion(+)
 create mode 100644 commands.txt

Pallavi Bandreddy@DESKTOP-POOHEON MINGW64 ~/gitcommands (master)
$ git status
On branch master
nothing to commit, working tree clean

Pallavi Bandreddy@DESKTOP-POOHEON MINGW64 ~/gitcommands (master)
$
```

```
Pallavi Bandreddy@DESKTOP-PO0HE0N MINGW64 ~/gitcommands (master)
$ git remote add origin https://github.com/Pallavibandreddy/gitcommand.git

Pallavi Bandreddy@DESKTOP-PO0HE0N MINGW64 ~/gitcommands (master)
$ git push -u origin master
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 231 bytes | 115.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Pallavibandreddy/gitcommand.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.

Pallavi Bandreddy@DESKTOP-PO0HE0N MINGW64 ~/gitcommands (master)
$ git status
On branch master
Your branch is up to date with 'origin/master'.

nothing to commit, working tree clean

Pallavi Bandreddy@DESKTOP-PO0HE0N MINGW64 ~/gitcommands (master)
$ git log
commit 1aadc9f7b37f6f2166661ee1760e87c05a67262c (HEAD -> master, origin/master)
Author: Pallavibandreddy <pallavibandreddy6@gmail.com>
Date:   Tue Feb 18 20:48:14 2025 +0530

    initial commit

Pallavi Bandreddy@DESKTOP-PO0HE0N MINGW64 ~/gitcommands (master)
$ git branch
* master

Pallavi Bandreddy@DESKTOP-PO0HE0N MINGW64 ~/gitcommands (master)
$ git checkout -d dev
fatal: git checkout: --detach does not take a path argument 'dev'

Pallavi Bandreddy@DESKTOP-PO0HE0N MINGW64 ~/gitcommands (master)
$ git checkout main
error: pathspec 'main' did not match any file(s) known to git

Pallavi Bandreddy@DESKTOP-PO0HE0N MINGW64 ~/gitcommands (master)
$ git checkout -b dev
Switched to a new branch 'dev'

Pallavi Bandreddy@DESKTOP-PO0HE0N MINGW64 ~/gitcommands (dev)
$ git checkout master
Switched to branch 'master'
Your branch is up to date with 'origin/master'.

Pallavi Bandreddy@DESKTOP-PO0HE0N MINGW64 ~/gitcommands (master)
$
```

```
Pallavi Bandreddy@DESKTOP-PO0HE0N MINGW64 ~/gitcommands (master)
$ git remote add origin https://github.com/Pallavibandreddy/gitcommand.git

Pallavi Bandreddy@DESKTOP-PO0HE0N MINGW64 ~/gitcommands (master)
$ git push -u origin master
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 231 bytes | 115.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Pallavibandreddy/gitcommand.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.

Pallavi Bandreddy@DESKTOP-PO0HE0N MINGW64 ~/gitcommands (master)
$ git status
On branch master
Your branch is up to date with 'origin/master'.

nothing to commit, working tree clean

Pallavi Bandreddy@DESKTOP-PO0HE0N MINGW64 ~/gitcommands (master)
$ git log
commit 1aadc9f7b37f6f2166661ee1760e87c05a67262c (HEAD -> master, origin/master)
Author: Pallavibandreddy <pallavibandreddy6@gmail.com>
Date:   Tue Feb 18 20:48:14 2025 +0530

    initial commit

Pallavi Bandreddy@DESKTOP-PO0HE0N MINGW64 ~/gitcommands (master)
$ git branch
* master

Pallavi Bandreddy@DESKTOP-PO0HE0N MINGW64 ~/gitcommands (master)
$ git checkout -d dev
fatal: git checkout: --detach does not take a path argument 'dev'

Pallavi Bandreddy@DESKTOP-PO0HE0N MINGW64 ~/gitcommands (master)
$ git checkout main
error: pathspec 'main' did not match any file(s) known to git

Pallavi Bandreddy@DESKTOP-PO0HE0N MINGW64 ~/gitcommands (master)
$ git checkout -b dev
Switched to a new branch 'dev'

Pallavi Bandreddy@DESKTOP-PO0HE0N MINGW64 ~/gitcommands (dev)
$ git checkout master
Switched to branch 'master'
Your branch is up to date with 'origin/master'.

Pallavi Bandreddy@DESKTOP-PO0HE0N MINGW64 ~/gitcommands (master)
$ git stash
No local changes to save

Pallavi Bandreddy@DESKTOP-PO0HE0N MINGW64 ~/gitcommands (master)
$
```